



Modelling and optimization of distributed assembly hybrid flowshop scheduling problem with transportation resource scheduling

Qiang Luo^a, Qianwang Deng^{a,*}, Xin Guo^a, Guiliang Gong^b, Xinhe Zhao^c, Liangwu Chen^c

^a State Key Laboratory of Advanced Design and Manufacturing for Vehicle Body, Hunan University, Changsha 410082, China

^b Department of Mechanical and Electrical Engineering, Central South University of Forestry and Technology, Changsha 410004, China

^c China Railway Engineering Equipment Group Technical Service Co., Ltd., Zhengzhou 450003, China

ARTICLE INFO

Keywords:

Distributed assembly hybrid flowshop scheduling
Transportation resource
Early assembly
MOEA/D
Problem-based local search

ABSTRACT

In the distributed assembly flowshop scheduling problem (DAFSP), the transportation resources between the production and assembly stages are rarely scheduled. However, for production managers, the transportation stage is important as it directly influences the cooperation between the production and assembly phases. Hence, in this study, a critical variant of the DAFSP, called the distributed assembly hybrid flowshop scheduling problem with transportation resource scheduling (DAHFSPT), is evaluated. DAHFSPT considers several vehicles for component delivery, and the early assembly is allowed in the assembly site. A mixed-integer linear programming model of the problem is developed to minimize the makespan and total energy consumption simultaneously. To solve the problem, an improved multi-objective evolutionary algorithm based on decomposition (IMOEA/D) is proposed. In the IMOEA/D, a well-designed encoding method, and eight heuristics decoding methods are embedded into the initialization operator to generate a high-quality initial population. A problem-based local search operator is presented to enhance the performance of the algorithm and accelerate its convergence. In addition, an elimination-reinitialization operator is designed to avoid optimal local solutions. A total of 200 instances based on classical benchmarks are formulated to verify the effectiveness of each improvement component in the IMOEA/D. Comprehensive experiments indicate that the IMOEA/D outperforms five well-known multi-objective algorithms concerning solution quality and distribution in solving the DAHFSPT.

1. Introduction

The assembly flowshop scheduling problem (AFSP), a classical combinatorial problem of assembly and flowshop scheduling, has been widely investigated in production systems, such as engine assembly (Lee et al., 1993). Under globalization, the AFSP in a distributed manufacturing setting (called distributed assembly flowshop scheduling problem, DAFSP) has attracted increasing attention compared to the traditional AFSP because it can respond faster to changing market demands (Pourhejazy et al., 2021; Zhao et al., 2021). The DAFSP includes the production and assembly stages and consists of two sub-problems: (1) assigning jobs to factories and (2) sequencing operations in each factory. Furthermore, research on DAFSP generally assumes that the assembly starts when all product parts are completed. However, in practice, an additional sub-problem needs to be solved: assigning vehicles for the transportation of completed jobs from the production to

assembly stages. Moreover, considering a limited workspace, product assembly may be performed in advance following a determined assembly sequence. This requires a balance among production, transportation, and assembly activities to realize smooth operation. Prior studies on DAFSP rarely consider transportation and early assembly (Huang et al., 2021; Song and Lin, 2021), which may cause a disorganized production owing to an uncoordinated transportation plan.

To coordinate transportation and assembly effectively, previous studies integrated the just-in-time (JIT) transportation method into the production plan using problems related to parallel machine scheduling (Jolai et al., 2011), distributed flowshop scheduling (Xiong et al., 2021), flowshop scheduling in a single plant (Elalouf, 2021), job shop scheduling (Ahmadian et al., 2021) and assembly line scheduling (Zhang et al., 2021). These studies have demonstrated that integration of the JIT approach into a transportation plan has advantages, including lower production costs, simplified workspaces, appropriate inventory levels,

* Corresponding author.

E-mail addresses: qiangluo@hnu.edu.cn (Q. Luo), deng_arbeit@hnu.edu.cn (Q. Deng), gotham@hnu.edu.cn (X. Guo), gongguiliang@hnu.edu.cn (G. Gong), zhaoxinhe771102@163.com (X. Zhao), nkdclw@163.com (L. Chen).

<https://doi.org/10.1016/j.cie.2023.109717>

Received 21 March 2023; Received in revised form 30 August 2023; Accepted 28 October 2023

Available online 31 October 2023

0360-8352/© 2023 Elsevier Ltd. All rights reserved.

and shorter production cycles. However, to the best of the authors' knowledge, integration of the JIT approach with the DAFSP has not been reported.

Based on this context, a new variant of the DAFSP, called the distributed assembly hybrid flowshop scheduling problem with transportation resource scheduling (DAHFSPT), is investigated in this study. In the DAHFSPT, the production, transportation, and assembly stages are considered simultaneously. The production stage includes several factories equipped with a hybrid flowshop; the transportation stage includes several vehicles, and the assembly stage involves a factory with an assembly machine. The early assembly is considered in the assembly site. A mixed-integer programming model (MILP) is established to minimize the makespan and total energy consumption simultaneously. Considering the facts that lots of variants of the multi-objective evolutionary algorithm based on decomposition (MOEA/D) have been designed to solve shop scheduling problems successfully (Zhou & Wu, 2020), an improved MOEA/D (IMOE/D) algorithm is proposed in this study to solve the problem. Unlike other MOEA/Ds, the proposed IMOE/D designs a four-layer chromosome encoding and develops eight heuristic methods to generate a high-quality population. The JIT strategy is embedded into chromosome decoding to bridge the production and assembly stages and reduce energy consumption in the transportation. Subsequently, a problem-based local search (PLS) operator is developed to improve local search ability of the algorithm. In addition, an elimination-reinitialization (ER) operator is proposed to remove redundant individuals and avoid optimal local solutions.

Considering that the proposed design is novel, no benchmarks in the literature can be used to verify the performance of the proposed algorithm. Thus, based on the benchmarks of the flowshop scheduling problem (Taillard, 1993), 200 instances classified as 20 combinations are formulated to test the performance of the IMOE/D. In addition, the Taguchi method of design of experiments (DOE) (Montgomery, & DouglasC, 2008) is used for the parameter calibration of the IMOE/D. The effectiveness of the designed IMOE/D components is verified through a set of experiments, and the IMOE/D is compared with five well-known algorithms to verify its superiority in solving the DAHFSPT.

The remainder of this paper is organized as follows. The background and related works are analyzed in Section 2. The problem description and model are presented in Section 3. The details of the IMOE/D are described in Section 4. The experiment and discussion are presented in Section 5. Finally, conclusions and future studies are presented in Section 6.

2. Related work

Over the past decade, plentiful studies on DAFSP have been reported. These studies can be categorized mainly based on problem models and optimization algorithms, as shown in Table 1.

Five variants of the DAFSP were studied based on problem models. Hatami et al. (2013) first defined the distributed assembly permutation flowshop scheduling problem (DAFPSP) and built its mixed-integer linear programming model (MILP) to minimize the makespan. The model considered several factories in the production stage and one machine in the assembly stage. Several studies based on this model were reported subsequently. Lu et al. (2022) solved energy-efficient DAFSP with limited buffers. Hatami et al. (2015); Huang and Gu (2021); Song and Lin (2021) discussed the DAFSP with sequence-dependent setup times. Second, Xiong and Xing (2014) investigated another variant of the DAFSP, called the distributed two-stage assembly scheduling problem (DTSASP), which involved the production and assembly stages in each factory. Two objectives were optimized simultaneously in their research. Deng et al. (2016); Lei et al. (2021); Lei and Wang (2020); Pourhejazy et al. (2021); Zhang and Xing (2018) discussed static DTSASP. Cai et al. (2020) solved the DTSASP with uncertain processing time. Furthermore, several researchers studied new DAFSP models by integrating general constraints in classical flowshop scheduling into the

Table 1

The related papers on DAFSP.

Author and Year	Problem Type	Objective	Method
Zhang et al. (2022)	Distributed assembly permutation flowshop scheduling problem	Makespan and total carbon emission	EDA
Huang et al. (2021)		Total flowtime	IG
Hamzadayi et al. (2021)		Makespan	Single seekers society algorithm
Zhang et al. (2021)		Makespan	EDA
Song and Lin (2021)		Makespan	HH
Huang and Gu (2021)		Makespan	BOA
Zhang et al. (2020)		Makespan	GA
Pan et al. (2019)		Makespan	Heuristic, VNS and IG
Lin et al. (2017)		Makespan	HH
Wang and Wang (2016)		Makespan	EDA
Lin and Zhang (2016)		Makespan	BOA
Hatami et al. (2015)		Makespan	Heuristic, VNS and IG
Hatami et al. (2013)		Makespan	Heuristic and VND
Pourhejazy et al. (2022)	Distributed two-stage assembly flowshop scheduling problem	Makespan	IG
Pourhejazy et al. (2021)		Makespan	IG
Lei et al. (2021)		Makespan	Cooperated teaching-learning-based optimization
Lei and Wang (2020)		Makespan and number of tardy jobs	Shuffled frog-leaping algorithm
Cai et al. (2020)		Total agreement index and fuzzy Makespan	VNS
Zhang and Xing (2018)		Total flowtime	Social spider optimization
Deng et al. (2016)		Makespan	MA
Wang et al. (2023)		Makespan and total tardiness	ABC
Wang et al. (2022)		Total tardiness	ABC
Lei and Li (2022)		Makespan and total energy consumption	Imperialist competitive algorithm
Zhao, et al. (2022)		Makespan and total energy consumption	Brain storm optimization algorithm
Zhao, et al. (2022)		Total tardiness	Water wave optimization algorithm
Niu et al. (2022)		Makespan and total energy consumption	Non-dominated sorting genetic algorithm II
Yang and Li (2022)		Makespan	Constructive heuristic algorithm
Li et al. (2022)		Total tardiness	IG
Huang et al. (2022)		Makespan	IIS

(continued on next page)

Table 1 (continued)

Author and Year	Problem Type	Objective	Method
Zhao et al. (2021)		Makespan	Water wave optimization algorithm
Zhao et al. (2021)		Makespan	Backtracking search algorithm
Li et al. (2021)		Weighted value of the makespan and the energy consumptions	Whale optimization algorithm
Shao et al. (2020)		Makespan	ILS
Shao et al. (2019)		Makespan	ILS, VNS
Li et al. (2017)		Makespan	ILS
Ji et al. (2016)		Makespan	HPSO
Xiong and Xing (2014)		Makespan and mean completion time	VNS and GA-VNS
Wang and Wang (2022)	Distributed flexible assembly flowshop scheduling problem	Total tardiness and total energy consumption	MA
Yang and Xu (2021)		Total cost	Heuristic, VND, IG
Zhang et al. (2020)		Makespan	MA
Zhang et al. (2018)		Makespan	Heuristic, VNS and PSO
Jia et al. (2023)	Distributed assembly hybrid flowshop scheduling problem	Makespan	MA
Lei and Dai (2023)		Makespan	Shuffled frog-leaping algorithm
Cai et al. (2022)		Makespan	Shuffled frog-leaping algorithm

Note: EDA (estimation of distribution algorithm), HH (hyper-heuristic), BOA (biogeography-based optimization algorithm), IG (iterated greedy algorithm), VNS (variable neighborhood search algorithm), VND (variable neighborhood descent algorithm), MA (memetic algorithm), ILS (iterated local search), PSO (particle swarm optimization), ABC (artificial bee colony Algorithm).

DAFSP. Ji et al. (2016); Li et al. (2017); Zhao et al. (2021) considered the DAFSP with a no-wait constraint. Shao et al. (2019); Zhao et al. (2021) studied the DAFSP with a no-idle constraint, and Shao et al. (2020) discussed the DAFSP with a blocking constraint (DABFSP). In addition, Yang and Xu (2021); Zhang et al. (2018); Zhang et al. (2020) focused on the distributed flexible assembly flowshop scheduling problem (DFAFSP) considering many machines in the assembly stage. Lastly, (Cai et al., 2022; Jia et al., 2023; Lei and Dai, 2023) examined a distributed assembly hybrid flowshop scheduling problem, in which the fabrication, transportation and assembly are equipped in each factory. They developed two shuffled frog-leaping algorithms and a multi-population memetic algorithm to solve their studied problem.

Owing to the NP-hard nature of DAFSP (as verified by Lee et al. (1993)), multiple heuristic and metaheuristic algorithms have been proposed to solve it. Pan et al. (2019) and Shao et al. (2020) studied DAFSP and DABFSP, respectively, using heuristics. Yang and Xu (2021); Zhang et al. (2018) designed heuristic algorithms to solve the DFAFSP. Many researchers solved DAFSPs using metaheuristic algorithms. Hatami et al. (2013); Yang and Xu (2021) proposed multiple variable neighborhood descent algorithms to solve the DAFSP. Song and Lin (2021); Zhang et al. (2020) handled the problem based on genetic algorithms. Huang et al. (2021); Yang and Xu (2021) dealt with the problem using iterated greedy algorithms. Wang and Wang (2016); Zhang et al. (2021) addressed the problem by designing the estimation of distribution algorithms. In addition, some rarely used algorithms enhanced the approach diversity in solving DAFSP, such as the imperialist competitive algorithm (Li et al., 2021), water wave optimization

algorithm (Zhao et al., 2021), teaching-learning-based optimization algorithm (Lei et al., 2021), and memetic social spider optimization algorithm (Zhang and Xing, 2018).

MOEA/D is a well-known population-based metaheuristic algorithm that has been widely applied to solve various multi-objective optimization problems (Danisa Romero-Ocano et al., 2022; Zhang et al., 2022). Jiang and Wang (2019) studied the energy-efficient hybrid flowshop scheduling problem with sequence-dependent setup time and designed MOEA/D to solve it. Jiang and Zhang (2019) solved the energy-oriented hybrid flowshop scheduling problem with limited buffers by developing an MOEA/D to minimize the total weighted tardiness and non-processing energy. MOEA/D was also used to solve the flowshop scheduling problem in a distributed environment (Wang et al., 2021). In addition, Zhao et al. (2017) solved the multi-objective job shop scheduling problem, and Wu et al. (2021) discussed the flexible job shop scheduling problem. These shop scheduling problems were solved using multiple variants of MOEA/D. Furthermore, Jiang et al. (2020) modified the MOEA/D to solve the distributed job-shop scheduling problem.

The above discussion can be summarized as follows: (1) The DAFSP has attracted considerable attention owing to its important role in the distributed production system. (2) The existing DAFSPs generally assume that the assembly process begins only when all components of a product have reached the assembly workshop, however, the early assembly may be crucial for an assembly site with limited working space in real-world production. (3) Almost all studies considering production, transportation and assembly stages, handle the transportation constraints by heuristic, while the precise vehicle scheduling for components transportation is critical when two or more objectives are simultaneously considered in the DAHFSPT systems. (4) MOEA/D has been proved to be effective in solving various shop scheduling problems, but it has not been used to solve the DAFSP. Thus, this study explores the DAHFSPT with transportation resource scheduling and proposes an IMOEA/D to minimize the makespan and total energy consumption.

3. Proposed DAHFSPT

3.1. Problem description

Fig. 1 illustrates the DAHFSPT, which can be described as follows: n jobs with k stages need to be processed in f factories. Subsequently, the completed jobs are transferred to the assembly factory and finally assembled into the product. This problem involves the production, transportation, and assembly stages. In the production stage, each factory is considered a hybrid flowshop equipped with b machines ($b > 1$) in each processing stage. O_{ij} ($i = 1, \dots, n; j = 1, \dots, k$) denotes the j -th stage of job i . The processing time and energy consumption of O_{ij} in each factory differ and are known in advance. In the transportation stage, t vehicles are used to transfer the completed job P_i (also called part P_i) from the production site to the assembly factory, and the transportation of part P_i starts once the allocated vehicle is available. Each vehicle travels to the location of part P_i in the empty state and delivers part P_i to the assembly factory in the loaded state, and only one part is transferred at a time. Different unit transportation speed and energy consumption of vehicles are considered, and the energy consumption of the vehicles in the loaded state is larger than that in the empty state. In the assembly stage, the parts that arrive at the assembly factory are assembled on a certain machine M_A according to the predetermined assembly sequence. It is noteworthy that the assembly of a product starts if a required part in the known assembly sequence is prepared and the assemble machine is idle.

In this study, the two objectives are to minimize the makespan and total energy consumption simultaneously. In this regard, the following decisions are made:

- (1) decision of factory assignment;
- (2) decision of operation sequence during the production phase;

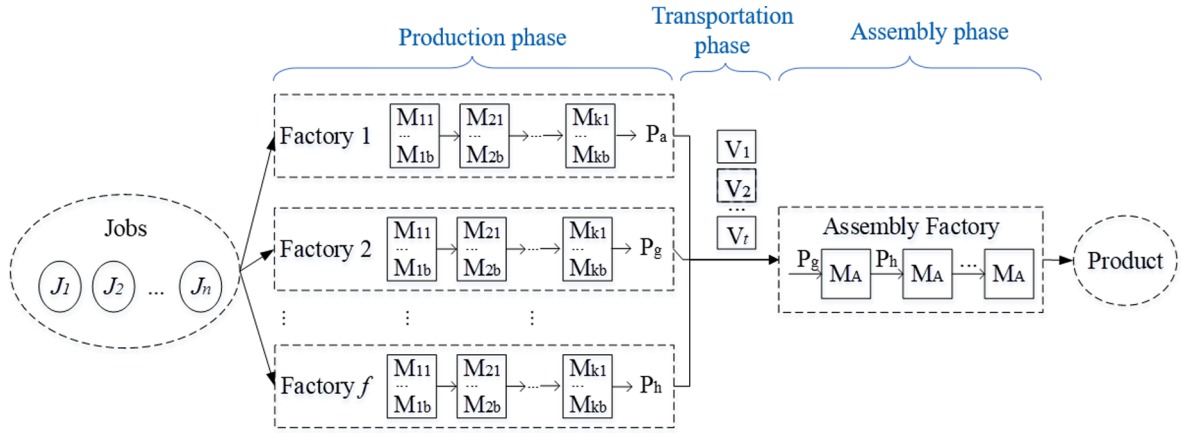


Fig. 1. Description of DAHFSPT.

- (3) decision of machine for each operation during the production phase;
- (4) decision of vehicle for each completed job during the transportation phase.

3.2. Notation

The notations used in the model are given as follows:

Indices	
p	Index of factory, $p = 1, 2, \dots, f$
k	Index of machine, $k = 1, 2, \dots, m_j$
i, g	Index of job, $i, g = 1, 2, \dots, n$, where $i \neq g$
j	Index of stage, $j = 1, 2, \dots, O(i)$
v	Index of vehicle, $v = 1, 2, \dots, t$
Parameters	
f	Total number of factories
m_j	Total number of machines of stage j in each factory
n	Total number of jobs
$O(i)$	Total number of operations for job i
t	Total number of vehicles
dis_p	Distance between the factory p and the assembly factory
T_{pijk}^{pro}	Processing time of operation O_{ij} on machine k in factory p
US_v^{loaded}	Unit speed of vehicle v in the loaded state
$US_v^{unloaded}$	Unit speed of vehicle v in the empty state
T_i^{ass}	Assembly time of job i
A_{ig}	Equals to 1 if part g is the precedent job of part i during assembly and otherwise equals to 0
S_{ij}^{pro}	Start time of operation O_{ij} during the production period
S_i^T	Start time of job i during the transportation period
S_i^A	Start time of job i during the assembly period
C_{ij}^{pro}	Completion time of operation O_{ij} during the production period
C_i^T	Completion time of job i during the transportation period
C_i^A	Completion time of job i during the assembly period
E_{pijk}^{pro}	Unit processing energy consumption of machine k of stage j in factory p during the production period
E_{pijk}^{idle}	Unit idle energy consumption of machine k of stage j in factory p during the production period
E_v^{loaded}	Energy consumption of unit distance of vehicle v in the loaded state
$E_v^{unloaded}$	Energy consumption of unit distance of vehicle v in the empty state
E^{ass}	Unit processing energy consumption during assembly
E^{idle}	Unit idle energy consumption during assembly
EC_P	Total energy consumption in the production period
EC_T	Total energy consumption in the transportation period
EC_A	Total energy consumption in the assembly period
TEC	Total energy consumption
M	A large positive number
Decision variables	
x_{pi}	Equals to 1 if job i is allocated to factory p and otherwise equals to 0
x_{pijk}	Equals to 1 if operation O_{ij} is processed on machine k in factory p and otherwise equals to 0
y_{pigj}	Equals to 1 if operation O_{gj} is the precedent operation of O_{ij} in factory p and otherwise equals to 0

(continued on next column)

(continued)

Indices	
y_{piv}	Equals to 1 if part i is transported by vehicle v from factory p and otherwise equals to 0
z_{igv}	Equals to 1 if part g is the precedent part of part i transported by vehicle v and otherwise equals to 0

3.3. MILP model

In order to make the model more reasonable, some assumptions are made as follows.

- No machine or vehicle can be suspended after starting.
- The setup time and corresponding energy consumption of the whole phases are dismissed.
- Each machine in production can only handle one job and each vehicle can only transport one part at a time.
- All basic information about the factories, machines and vehicles, such as processing time and unit energy consumption, are known in advance.
- The speed of each vehicle is constant and known in advance, and each job must be transferred once.
- The assembly sequence of each product is predetermined.
- If the assembly machine is idle and the jobs obeying to the assembly sequence has arrived, the assembly will begin.

Based on the information above, the MILP model can be formulated as follows.

Objective function:

$$\min(\text{makespan}, \text{TEC}) \quad (1)$$

where

$$\text{makespan} = \max\{C_i^A\}, \forall i \in [1, 2, \dots, n] \quad (2)$$

$$\text{TEC} = EC_P + EC_T + EC_A \quad (3)$$

$$EC_P = \sum_{p=1}^f \sum_{i=1}^n \sum_{j=1}^{O(i)} \sum_{k=1}^{m_{pj}} x_{pijk} T_{pijk}^{pro} E_{pijk}^{pro} + \sum_{p=1}^f \sum_{i=1}^n \sum_{g=1}^n \sum_{j=1}^{O(i)} \sum_{k=1}^{m_{pj}} y_{pigj} (S_{gj}^{pro} - C_{ij}^{pro}) E_{pijk}^{idle} \quad (4)$$

$$EC_T = \sum_{v=1}^t \sum_{i=1}^n \sum_{p=1}^f y_{piv} (E_v^{loaded} + E_v^{unloaded}) dis_p \quad (5)$$

$$EC_A = \sum_{i=1}^n E_i^{ass} T_i^{ass} + \sum_{i=1}^n \sum_{g=1}^n A_{ig} (S_g^A - C_i^A) E^{idle} \quad (6)$$

Constraints:

Each job must be processed in a factory.

$$\sum_{p=1}^f x_{pi} = 1 \forall i \in [1, 2, \dots, n] \quad (7)$$

Each operation is processed once in the allocated factory.

$$\sum_{k=1}^{m_j} x_{pijk} = x_{pi} \forall i \in [1, 2, \dots, n], j \in [1, 2, \dots, O(i)], p \in [1, 2, \dots, f] \quad (8)$$

Each machine can only process one operation at a time.

$$y_{pigj} + y_{pgij} \leq 1 \quad (9)$$

$$y_{pigj} + y_{pgij} \geq x_{pijk} + x_{pgjk} - 1 \quad (10)$$

$$S_{gj}^{pro} \geq S_{ij}^{pro} + x_{pijk} T_{pijk}^{pro} - M(3 - x_{pijk} - x_{pgjk} - y_{pigj})$$

$$\forall i, g \in [1, 2, \dots, n], j \in [1, 2, \dots, O(i)], p \in [1, 2, \dots, f], k \in [1, 2, \dots, m_{pj}] \quad (11)$$

Each operation subjects to sequence constraint.

$$S_{i(j+1)}^{pro} \geq S_{ij}^{pro} + x_{pijk} T_{pijk}^{pro} \forall i \in [1, 2, \dots, n], j \in [1, 2, \dots, O(i) - 1], p \in [1, 2, \dots, f], k \in [1, 2, \dots, m_{pj}] \quad (12)$$

Each job must be transported once from the allocated factory to the assembly factory.

$$\sum_{p=1}^f \sum_{v=1}^t y_{piv} = 1 \forall i \in [1, 2, \dots, n] \quad (13)$$

$$\sum_{v=1}^t y_{piv} = x_{pi} \forall i \in [1, 2, \dots, n], p \in [1, 2, \dots, f] \quad (14)$$

Start time of each job in transportation should be later than their completion time in the production period.

$$S_i^T \geq C_{ij}^{pro} + \frac{dis_p}{US_v^{unloaded}} - M(1 - y_{piv})$$

$$\forall i \in [1, 2, \dots, n], j \in [1, 2, \dots, O(i)], p \in [1, 2, \dots, f], v \in [1, 2, \dots, t] \quad (15)$$

Each vehicle can only deliver a part at a time.

$$S_g^T \geq S_i^T + \frac{dis_p}{US_v^{loaded}} + \frac{dis_p'}{US_v^{unloaded}} - M(3 - y_{piv} - y_{p'gv} - z_{igv})$$

$$\forall i, g \in [1, 2, \dots, n], p, p' \in [1, 2, \dots, f], v \in [1, 2, \dots, t] \quad (16)$$

The transportation of components is subjected to the transportation sequence.

$$\sum_{v=1}^t z_{igv} \leq 1 \wedge \sum_{v=1}^t z_{g'iv} \leq 1, \forall i, g \in [1, 2, \dots, n], i \neq g \quad (17)$$

$$z_{igv} + z_{g'iv} \geq y_{piv} + y_{p'gv} - 1, \forall i, g \in [1, 2, \dots, n], i \neq g, v \in [1, 2, \dots, t], p, p' \in [1, 2, \dots, f], p \neq p' \quad (18)$$

Start time of each job in assembly should be later than their completion time in the transportation period.

$$S_i^A \geq C_i^T \forall i \in [1, 2, \dots, n] \quad (19)$$

The components are assembled following the predetermined sequence.

$$S_g^A \geq C_i^A - M(1 - A_{ig}) \forall i, g \in [1, 2, \dots, n] \quad (20)$$

4. Proposed IMOEA/D

Considering the solving complexity of the proposed DAHFSPT, this section aimed to design an effective algorithm called IMOEA/D to solve the problem. The new algorithm extended the classical MOEA/D proposed by Zhang and Li (2007), and incorporated some problem-specific operators. In detail, the chromosome encoding and decoding methods, population initialization, adaptive crossover and mutation operators, problem-based local search and elimination-reinitialization operator were elaborated sequentially. Finally, we introduced the framework of the proposed method.

4.1. Chromosome encoding and decoding

4.1.1. Encoding

As for the four decisions described in the previous section, a four-layer array representation method was designed for solutions, comprising the factory selection (FS), machine selection (MS), operating sequence (OS), and transportation equipment selection (TS) arrays. The FS and TS arrays were arranged according to the job index. The OS array was ranked according to the operation sequence of each stage in each factory. The MS array was organized into all operations. An example is illustrated in Fig. 2, where five jobs with two stages, two available factories, two machines for each operation in each factory, and two vehicles are included.

In Fig. 2, the FS array [1 2 1 1 2] indicates that five jobs J1-J5 are processed in factory 1, 2, 1, 1 and 2, respectively. TS array [2 1 2 2 1] shows that these jobs are transported to the assembly factory by the vehicles 2, 1, 2 and 1, respectively. As for the production stage in each factory, the OS array [1 3 4 3 1 4 2 5 2 5] shows that the jobs 1, 3, 4 are handled in factory 1 with the sequence O11 → O31 → O41 in the stage 1 and O32 → O12 → O42 in the stage 2. The remaining jobs 2, 5 are processed in factory 2 with the sequence O21 → O51 in the stage 1 and O22 → O52 in the stage 2. Regarding the MS array [1 2 2 1 2 2 1 1 2 2], the first number 1 (corresponding to O11) represents the first machine in the set of available machines in the stage 1, and its second number 2 (corresponding to O12) denotes the second machine in the set of available machines in the stage 2. The meaning of other numbers in the MS array is similar with those in the operations O11 and O12.

4.1.2. Decoding with JIT strategy

When the chromosome encoding was determined, the general active scheduling decoding method described in Han et al. (2021) was employed to decode individuals. The main procedures were described below. Decide all assigned jobs in each factory according to the FS array, then for each factory p , obtain the permutation of its all assigned jobs based on the processing sequence in OS array and selected machines in MS array. Note that each operation starts as soon as possible according to the operation and machine constraints. Determine the transportation

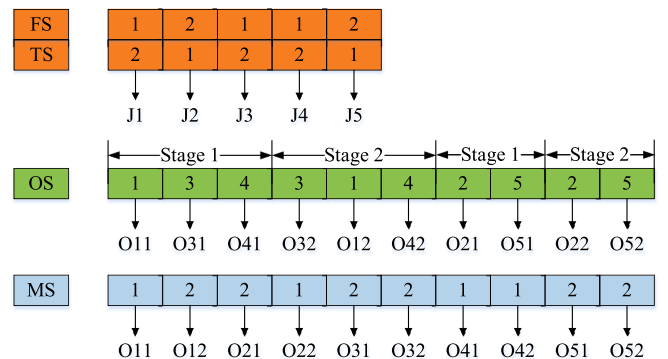


Fig. 2. Chromosome encoding.

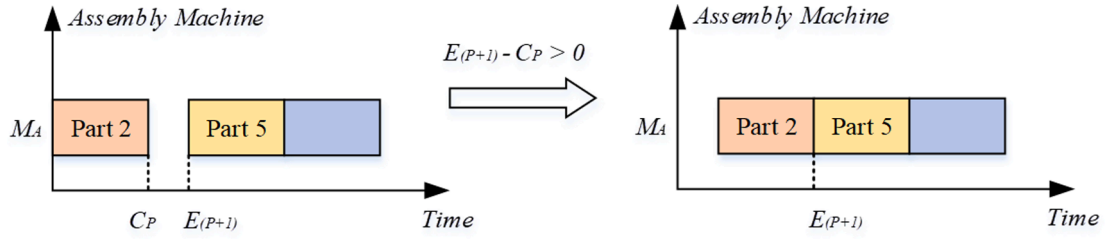


Fig. 3. Just-in-time strategy in transportation.

starting time and ending time of each part according to the chosen vehicle in TS array, and adopt the First Come First Serve (FCFS) rule when two or more jobs are ready for transportation. When a component meeting the product assembly sequence is delivered at the assembly factory and the assembly machine is available, the assembly is then carried out and the completion time of the component is determined. After all products have been finished, the makespan and total energy consumption can be computed.

Because of the asynchrony between transportation and assembly, an assembly machine can sometimes be idle. Ideally, the assembly machine functions continuously without waiting. This is possible via JIT transportation (Jayaswal and Agarwal, 2014), where jobs are delivered when they need to be assembled. An example is shown in Fig. 3. There is idle time between parts a and b. According to the JIT strategy, a better solution can be acquired by delaying the start time of Part a. Thus, the JIT strategy is adopted to further optimize total energy consumption by removing the idle energy consumption in the assembly for each solution, obtaining a smooth assembly plan. The JIT method involves the following steps:

- Step 1:** Extract the start and end times of all parts in the assembly plan.
- Step 2:** Save the idle time set t for any two adjacent parts and rank set t in descending order of the start time of the parts.
- Step 3:** Delay the transport start time of the previous part in the adjacent parts if the value in set t is greater than zero.

4.2. Population initialization

High-performance initial populations provide a good search starting point for population evolution, which in turn finds better Pareto front solutions. To achieve this goal, we designed some effective heuristic rules to determine the factory allocation, machine selection, and processing sequence arrays.

4.2.1. Job assignment

A critical decision in distributed scheduling problems is determining the factory assignment of jobs. Referring to the existing literature, the NR1 and NR2 rules suggested by Hatami et al. (2013) are effective for solving the DAFSP. The extended applications can be found in the reports by Shao et al. (2020); Yang and Xu (2021). The main idea of the NR1 rule is that a job is assigned to the factory with the lowest makespan without considering the processing time of the current job. In contrast, according to the NR2 strategy, a job is processed in the factory with the shortest makespan, considering the processing time of the current job. Given the objective of minimizing the makespan and total energy consumption, the NR2 method was used in this study to determine the FS array. To facilitate the calculation, the processing time of jobs in each stage was computed as the mean processing time of all available machines for each operation.

4.2.2. Heuristic rules

The list scheduling algorithm has been proven to be effective and is widely used in various hybrid flowshop scheduling problems. According

to Han et al. (2021), the OS and MS arrays within the factory can be determined using the steps enumerated below. Note that the classical heuristics FIM (Mainieri and Ronconi, 2013) and FEM (Han et al., 2021) rules correspond to the first idle of machine rule and the first ending of machine, respectively.

- (1) Randomly generate a job list with the same number of jobs to denote the operational sequence in the first stage.
- (2) Arrange the completion time of all operations in a non-decreasing order to determine the operation sequence of the next stage (seen as FCFS rule).
- (3) Calculate the machine priority for each operation according to FIM or FEM rules.
- (4) Select the machine with the highest priority for each operation.
- (5) Repeat the above procedures until the operations of all stages are scheduled.

Considering the features of early assembly, the solutions may be better when all jobs are completed in an order closer to the assembly sequence. Therefore, three problem-specific heuristics are proposed: FCFS-AS, AS, and AS-FCFS. Their main steps can be summarized in Algorithms 1–3 (the inputs $jsArray$, $asArray$, job_num , $stage_num$, ope_set denote the initial order of the allocated jobs in a factory, the standard assembly order, number of jobs assigned to this factory, stage number of jobs, and positional indices of all operations, respectively, and the outputs $chromOS$, $chromMS$ represent the processing sequence of the operations and their processing machines, respectively).

FCFS-AS rule: This method determines the operation sequence for each stage using the FCFS rule. However, the last stage uses the AS rule. This rule can help obtain a solution in which the manufacturing time is less than that of the FCFS rule.

AS rule: For a random job sequence in a factory, the processing sequence of all the jobs at each stage is always consistent with the assembly sequence.

AS-FCFS rule: The operation sequence of each stage is determined by the FCFS rule. However, the first stage uses the AS rule. Compared with the AS rule, this strategy eliminates the waiting time for all jobs at each stage.

Based on the FCFS and three heuristics above, eight methods were generated to determine the OS and MS arrays by combining them with the FIM and FEM rules. These heuristics were used to generate a high-quality population in the initialization, and their performance was tested based on a comparison with the classical heuristics and stochastic method.

Algorithm 1: FCFS-AS rule

Input: $jsArray$, $asArray$, job_num , $stage_num$, ope_set

Output: $chromOS$, $chromMS$

1. Set $chromOS$, $chromMS \leftarrow null$

2. **for** $stage \leftarrow 1$ to $stage_num$ **do**

3. **if** $stage == stage_num$ **do**

4. Update $jsArray$ by ranking all jobs of $jsArray$ according to the job sequence in $asArray$

5. **end if**

6. Update $chromOS$ by combining $chromOS$ with $jsArray$

(continued on next page)

(continued)

Algorithm 1: FCFS-AS rule

```

7. for  $j \leftarrow 1$  to  $job\_num$  do
8.    $job\_rank \leftarrow jsArray[j]$ 
9.   Find the position  $job\_pos$  of job  $job\_rank$  in  $ope\_set$ 
10.  Determine machine selection  $M_k^*$  by FIM or FEM rules
11.   $chromMS[job\_pos] \leftarrow M_k^*$ 
12. end for
13. Refresh  $jsArray$  by ranking all jobs in ascending order according to their
    completion time
14. end for
15. return ( $chromOS$ ,  $chromMS$ )

```

Algorithm 2: AS rule

Input: $jsArray$, $asArray$, job_num , $stage_num$, ope_set
Output: $chromOS$, $chromMS$

```

1. Update  $jsArray$  by ranking all jobs of  $jsArray$  according to the job sequence in
    $asArray$ 
2. Set  $chromOS$ ,  $chromMS \leftarrow null$ 
3. for  $stage \leftarrow 1$  to  $stage\_num$  do
4.   Update  $chromOS$  by combining  $chromOS$  with  $jsArray$ 
5.   for  $j \leftarrow 1$  to  $job\_num$  do
6.      $job\_rank \leftarrow jsArray[j]$ 
7.     Find the position  $job\_pos$  of job  $job\_rank$  in  $ope\_set$ 
8.     Determine machine selection  $M_k^*$  by FIM or FEM rules
9.      $chromMS[job\_pos] \leftarrow M_k^*$ 
10.  end for
11. end for
12. return ( $chromOS$ ,  $chromMS$ )

```

Algorithm 3: AS-FCFS rule

Input: $jsArray$, $asArray$, job_num , $stage_num$, ope_set
Output: $chromOS$, $chromMS$

```

1. Update  $jsArray$  by ranking all jobs of  $jsArray$  according to the job sequence in
    $asArray$ 
2. Set  $chromOS$ ,  $chromMS \leftarrow null$ 
3. for  $stage \leftarrow 1$  to  $stage\_num$  do
4.   Update  $chromOS$  by combining  $chromOS$  with  $jsArray$ 
5.   for  $j \leftarrow 1$  to  $job\_num$  do
6.      $job\_rank \leftarrow jsArray[j]$ 
7.     Find the position  $job\_pos$  of job  $job\_rank$  in  $ope\_set$ 
8.     Determine machine selection  $M_k^*$  by FIM or FEM rules
9.      $chromMS[job\_pos] \leftarrow M_k^*$ 
10.  end for
11. Refresh  $jsArray$  by ranking all jobs in ascending order according to their
    completion time
12. end for
13. return ( $chromOS$ ,  $chromMS$ )

```

4.3. Global search operators

4.3.1. Crossover

In the evolutionary stage, four crossovers based on a binary matrix are proposed for offspring generation. For each array in the chromosome, a binary matrix of corresponding size is randomly generated, where the positions with the number 1 in the binary matrix represent the crossover points. The crossover on the FS, TS, and MS arrays is executed on crossover points by directly swapping the corresponding chromosome segments in the two parents. The crossover on OS array is performed by operation swapping using the OBC method (Gong et al., 2020). Four examples are provided below.

Fig. 4 (a)–(b) show the crossover methods on FS, TS and MS arrays. The crossover points are performed at position [1, 3, 5] on the FS array, position [2, 4] on the TS array and position [1, 3, 4, 8] on the MS array. The offspring C1 and C2 can be generated by directly swapping the genes of crossover points. Fig. 4 (c) illustrates the crossover on the OS array, in which the crossover is performed at position [2, 3, 5, 7, 8]. Taking the child C1 as an example, a new child can be generated by the following steps: extract the genes on the crossover points from P1, and here is [3 4 1 2 5]; classify the information of [3 4 1 2 5] as [031 041

011 021 051]; then find these operations on the OS array of P2, namely, position [1 2 3 7 8]; finally copy the OS array of P2 to C1 and replace the genes on position [1 2 3 7 8] of C1 with the operation [031 041 011 021 051].

4.3.2. Mutation

A mutation operator is generally used to obtain new solutions by slightly perturbing the chromosome. Mutations on the OS, MS, and TS arrays are carried out by the two-point mutation method. Two positions are determined randomly on the chromosome, and the genes at these positions are exchanged. Considering the TS array [2 1 2 2 1] in Fig. 5 as an example, two random positions are determined as [2 4]. The TS array [2 2 2 1 1] of child C1 is obtained by exchanging the genes of the TS array at position [2 4]. The mutations in MS and OS are similar to those in TS, as shown in Fig. 5.

4.4. Problem-based local search operator

A local search operator is necessary to solve highly complex shop scheduling problems. In this study, four decisions were involved: job assignment, operation sequence, machine selection, and vehicle selection. Obtaining the optimal solution for many objectives simultaneously could be challenging. Additionally, it may be difficult to find the optimal solution using only evolutionary operators such as crossover and mutation. Consequently, a problem-based local search (PLS) operator was designed to enhance the searching ability of the IMOEA/D. Three search directions were considered in the PLS operator: altering the job assignment, adjusting the operation sequence, and changing the transportation vehicle.

Search direction 1: Alter factory assignment of a critical job. The factory assignment of jobs plays a critical role in the distributed scheduling problem because the remaining decisions are based on it. Furthermore, the makespan depends on the critical path; that is, the longest path without a gap between two adjacent activities of machines from the start to the end. All jobs on the critical path were considered critical jobs. A solution with a shorter makespan may be obtained by adjusting the factory assignments of critical jobs. Therefore, four methods based on critical jobs are proposed here: *insert_in_factory*, *insert_all_factory*, *swap_in_factory* and *swap_all_factory*.

insert_in_factory method: Generate a new individual by randomly selecting a critical job, inserting it into any possible position in a random factory, and returning the best solution.

insert_all_factory method: Generate a new individual by randomly selecting a critical job and inserting it into a random position in each factory once and return the best solution.

swap_in_factory method: Obtain a new solution by randomly selecting a critical job, swapping it with any job in a random factory, and returning the best solution.

swap_all_factory method: Obtain a new solution by randomly selecting a critical job and swapping it with a random job in each factory, and return the best solution.

Search direction 2: The operation sequence is adjusted. In the critical factory (i.e., the factory with the maximum completion time), the adjustment of the operation sequence can generate a shorter makespan. In the proposed method, the assembly sequence is pre-determined. The solutions can be optimized by closing the output order of the jobs in the production stage to the standard assembly sequence A_S . Thus, a method based on adjusting the operation sequence was proposed.

AOS method: The output order of all jobs in the production stage is sorted in ascending order, denoted as A_C . Subsequently, for each $j(j \in \{1, \dots, job_num\})$, if $A_C(j)$ and $A_S(j)$ are different, swap the corresponding jobs of $A_C(j)$ and $A_S(j)$ for each stage in the chromosome to obtain the neighborhoods. Finally, the new neighbor is returned if a new and better solution is generated.

Search direction 3: Tuning vehicle arrangement. The transportation

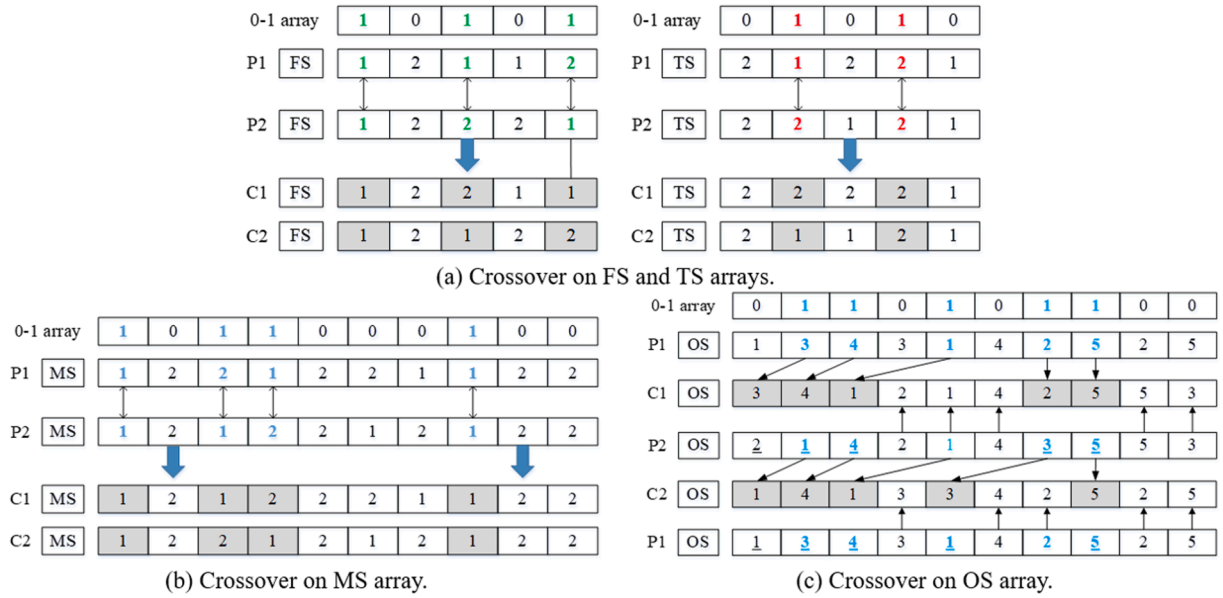


Fig. 4. Crossover operator.

stage can be optimized when the ending time of a job in the transportation stage and its start time in the assembly stage are different. A vehicle with lower energy consumption is selected to decrease the energy consumption without expanding the makespan.

TVA method: For a solution, the idle time of jobs between the ending time in the transportation phase and the start time in the assembly phase are listed. Subsequently, the vehicles of all jobs with idle time are changed into vehicles with lower energy consumption to obtain their neighborhoods. The best neighbor is returned if a new better solution is generated.

Based on the above three search directions, four methods can be combined. Direction 1 is used randomly for a time, and the **AOS** and **TVA** methods are applied each time. To make these methods more efficient, this the Meta-Lamarckian learning strategy proposed by Ong and Keane (2004) was adopted. In this strategy, the old solution is replaced by a new solution with better performance during the PLS operator, and the new individual participates in the remaining steps of the local search operation. The framework of the PLS operator is summarized in Fig. 6.

4.5. Elimination-reinitialization operator

Duplicate individuals in the population may be produced during initialization and evolutionary operators, which may reduce the population diversity and lead to the generation of local optimal solutions. To avoid this, an elimination-reinitialization (ER) operator was designed in

this study. The procedure for the ER operator is described as follows:

- Step 1:** For iteration i , remove the duplicates (solutions with same objective values) from population P .
- Step 2:** Calculate the mean objectives M_j ($j = 1, \dots, m$) of the optimal non-dominated individuals in P , and $S(i, j) = M$.
- Step 3:** If $i > 9$, verify the consistency of objective values in array S .
- Step 4:** If the values in S are the same, the 20 % worst solutions in P are removed, and the first row of S is deleted.
- Step 5:** Initialize $PS - ||P||$ (PS is the population size) solutions randomly, and add them into population P for the next generation.

4.6. Algorithm framework

Based on the above sections, the architecture of the proposed algorithm was given in Fig. 7. The detailed procedures were performed as follows.

Step 1: Initialization.

Step 1.1: Input initial parameters and a uniform spread of PS weight vectors $\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_{PS}\}$, set $PF = \phi$.

Step 1.2: Compute the Euclidean distances between any two weight vectors and then work out the T closest weight vectors to each weight vector. For each i , set $B(i) = \{i_1, \dots, i_T\}$, where $\lambda^{i_1}, \dots, \lambda^{i_T}$ are the closest weight vectors to λ^i .

Step 1.3: Generate an initial population by heuristics.

Step 1.4: Initialize $z^* = (z_1^{min}, \dots, z_m^{min})$ by getting the minimum from each target direction.

Step 2: Updating population. For $i = 1 : PS$, do.

Step 2.1: Randomly select two indexes from $B(i)$, then generate two new solutions from the selected individuals using global search operators, and remain the better one as x_{child} .

Step 2.2: Apply a problem-specific local search operator on x_{child} to produce x_{child}' .

Step 2.3: Update of z^* : For each $j = 1, \dots, m$, if $z_j > f_j(x_{child}')$, $z_j = f_j(x_{child}')$.

Step 2.4: Update of neighboring solutions: $j \in B(i)$, if $g^{te}(x_{child}' | \lambda_j, z) \leq g^{te}(x_j | \lambda_j, z)$ then set $x_j = x_{child}'$.

Step 2.5: Update of PF : Remove from all the individuals dominated by x_{child}' . Add x_{child}' to PF if no solutions in PF dominate x_{child}' .

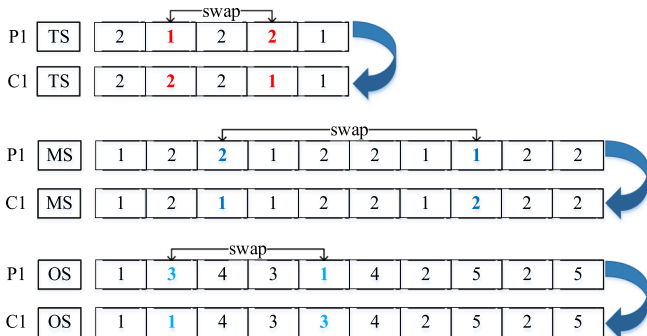


Fig. 5. Mutation operator.

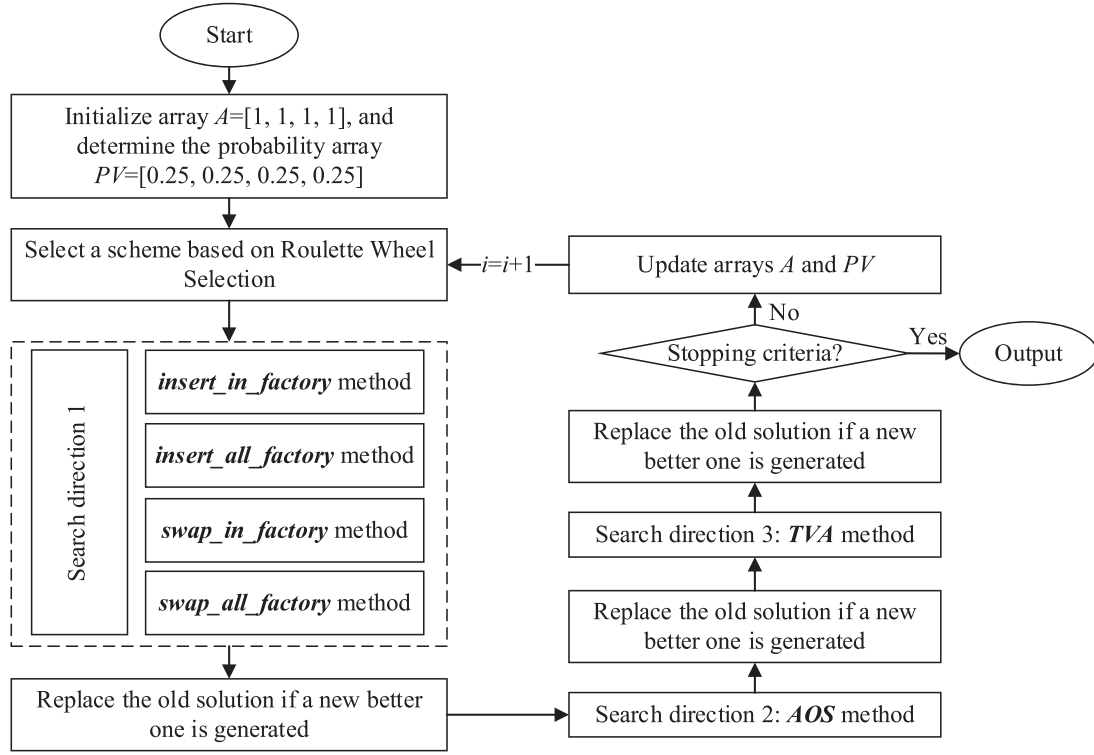


Fig. 6. Framework of local search operator.

Step 3: Avoiding local optimal solutions. Executing the ER operator if the PF remains unchanged for 10 iterations.

Step 4: Stopping criterion. If stopping criterion is satisfied, then stop and output PF. Otherwise, go to Step 2.

vehicles, respectively. Each instance was repeated 30 times to avoid randomness, and the best results were marked in **bold**.

5.1. Benchmark instance and performance evaluation

Referring to the classical benchmark in Taillard (1993), we developed 20 combination instances for this problem, where each combination contains 10 instances of the same scale. Specifically, they included 12 small-size combinations derived from Taillard tai0120 $\times$ 20–tai1020 $\times$ 20, and 8 large-size combinations generated from tai01100 $\times$ 10–tai10100 $\times$ 10. We set the basic data to be $n \in \{5, 10, 20\}$, $s \in \{5, 10\}$, $f \in \{2\}$, $e \in \{2, 3\}$, $t \in \{3\}$ in the small-scale instance, and $n \in \{50, 100\}$, $s \in \{5, 10\}$, $f \in \{4\}$, $e \in \{2, 3\}$, $t \in \{6\}$ in the large-size instances. The benchmark scale was marked as $n \times s \times f \times e \times t$ and all combinations

5. Experiment and discussion

To evaluate the effectiveness of our proposed algorithm, extensive experiments are carried out in this section. All algorithms were encoded in MATLAB R2016, and run on a PC equipped with an Intel Core i7 2.5 GHz and 16 GB RAM processor. In the experiments, the stopping criterion was set as $0.01 \times n \times s \times f \times e \times t$ (seconds), where n , s , f , e and t are the number of jobs, number of stages for each job, number of factories, number of available machines for each operation and number of

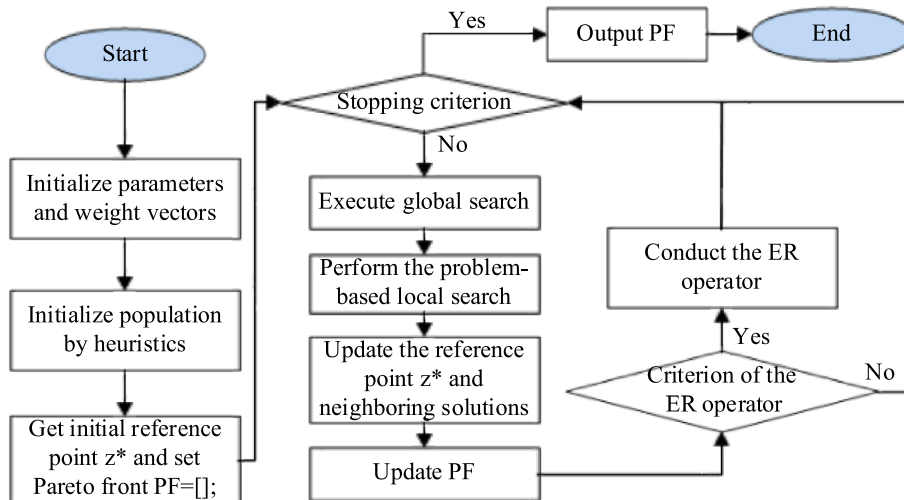


Fig. 7. Flowchart of the proposed IMOEA/D.

were denoted as C01-C20 in sequence. The details in benchmark construction are explained below.

In the production period, the processing time T_{pjk}^{pro} is produced obeying a normal distribution whose average value is T_{ij} and the standard deviation is $0.15 \times T_{ij}$ (Han, et al., 2021), where T_{ij} is the processing time of job i in j -th operation in the standard benchmark. The unit processing and idle energy consumption (E_{pk}^{pro} and E_{pjk}^{idle}) for each machine in the processing stage are generated randomly from (0.5, 1) and (0.2, 0.4), respectively. In the transportation stage, the unit speed and energy consumption of vehicles are produced randomly from $US_v^{loaded} \in (0.5, 1)$ and $E_v^{loaded} \in (1, 1.5)$ in loaded state, $US_v^{unloaded} \in (1, 1.5)$ and $E_v^{unloaded} \in (0.5, 1)$ in empty state, respectively. The distance dis_p between the factories of the production stage and the assembly factory is generated randomly from (0, 50), the position of the assembly factory is set as (0, 0), and the original position of all vehicles is (0, 0). In the assembly stage, the assembly time for each job is $T_i^{ass} = 2 \times T_{ij}$. The energy consumption of assembly machine is defined as 2 in working state and 1 in idle time.

In this paper, two well-known metrics were used to evaluate the algorithms' performance, namely, the Ratio of non-dominated solution (R_{nd}) and the Inverted Generation Distance (IGD) (Gao et al., 2014), which are regarded as classical indicators for solving the MOPs. At the same time, they have been widely used in various shop scheduling problems (Luo et al., 2020; Luo et al., 2023). Let X be the approximations of the Pareto front (PF) of the evaluated algorithm, P^* be the true PF obtained by the non-dominated sorting on the PF solutions of the all evaluated algorithms, and $|P^*|$ be the size of P^* .

R_{nd} metric: the metric denotes the ratio of the number of elements in $\{x|x \in X, x \in P^*\}$ to $|P^*|$. Obviously, the larger the R_{nd} is, the more PF solutions are generated, reflecting a better exploration ability.

IGD metric: the IGD value shows the distribution and diversity of the PF solutions. It can be computed by:

$$IGD(X, P^*) = \frac{1}{|P^*|} \sum_{x \in P^*} \min_{y \in X} d(x, y) \quad (19)$$

where $d(x, y)$ is the Euclidean distance between points x and y . Evidently, a smaller $IGD(X, P^*)$ corresponds to a closer distance between X and P^* , implying the better distribution and diversity. For the sake of fairness, the objectives of the PF solutions in X and P^* are normalized by the following formula.

$$\tilde{f}_i(x) = \frac{f_i(x) - f_i^{min}}{f_i^{max} - f_i^{min}}, \quad i = 1, 2, 3 \quad (20)$$

Where f_i^{min} and f_i^{max} are the minimum and maximum values of the i -th objective among all solutions, respectively.

5.2. Parameter calibration

Key parameters of the IMOEA/D directly influence its function in solving the problem. Here, we designed an experiment based on the Taguchi approach to tune these parameters, including: population size (PS), mutation rate (MR), local search rate (LSR), and neighbor size (T). In this experiment, each parameter contains four levels, that is, $PS \in \{30, 40, 50, 60\}$, $MR \in \{0.1, 0.15, 0.2, 0.25\}$, $LSR \in \{0.2, 0.3, 0.4, 0.5\}$ and $T \in \{5, 10, 15, 20\}$. Correspondingly, the orthogonal array L16 (4^4) is formulated. To make a trade-off, the combinations C07 (size: $10 \times 10 \times 2 \times 2 \times 3$) in the small-scale instances and C13 (size: $50 \times 5 \times 4 \times 2 \times 6$) in the large-scale instances were selected randomly for computation.

Fig. 8 showed the effect tendency of main parameters based on the average IGD values. In the picture, the minimum IGD values for PS, MR, LSR and T were generated by levels 3, 1, 2 and 4 in the combination C07, and levels 3, 2, 1 and 4 in the combination C13, respectively. As for PS and T, it can be concluded that $PS = 50$ and $T = 20$ were the best level in computation. However, in terms of MR and LSR, there were some

conflicts in two computations. Considering the IGD values in different parameter levers comprehensively, the balanced parameter combination may be determined as $MR = 0.1$ and $LSR = 0.2$. Thus, the parameter configuration of the IMOEA/D is determined as: $PS = 50$, $MR = 0.1$, $LSR = 0.2$ and $T = 20$.

5.3. Comparison between CPLEX and IMOEA/D in small instances

To validate the proposed model, we used the IBM ILOG CPLEX 12.9.0 solver to find optimal makespan solutions based on benchmarks C01-C08. The maximum runtime for each benchmark was set to 3600 s. We also tried to find the optimal TEC solutions by the CPLEX according to Equations (3). These solutions were compared with those acquired by the IMOEA/D, as shown in Table 2. In Table 2, “-” indicates that a solution cannot be obtained within 3600 s.

From Table 2, it is obvious that CPLEX obtained the optimal solutions in terms of the makespan on instances C01-C04. However, the CPLEX cannot find the optimal solutions within 3600 s in computing the combinations C05-C08. Our proposed IMOEA/D can solve all the tested instances within acceptable computational time. On the combinations C01-C04, although the CPLEX outperformed the IMOEA/D in terms of the makespan, our IMOEA/D was superior to the CPLEX in terms of the TEC. Nevertheless, we also noted that the solutions generated by the CPLEX and our IMOEA/D were non-dominated. Comparing the CPU time, the CPLEX was faster than the IMOEA/D on instances C01-C03, but IMOEA/D was faster than the CPLEX on examples C04-C08.

5.4. Comparison of proposed components in IMOEA/D

The performance of the proposed components in the IMOEA/D was compared through a set of comparative experiments, including comparisons of the initialization strategies, PLS, and ER operators. First, the performance of the proposed initialization method was tested by comparing it with two classical methods and a random method. Second, the effectiveness of the PLS operator was verified by comparing the IMOEA/D with and without the PLS operator. Finally, the performance of the ER operator was verified by comparing the IMOEA/D with and without the ER operator. Table 3 lists the detailed configurations of the IMOEA/D and its variants. In the experiments, 200 instances in 20 combinations were calculated under the same time-stopping criteria. The average of 10 instances was expressed as the final result of the corresponding combination. The results of the three comparative experiments based on the R_{nd} and IGD indicators are presented in Table 3 and Table 4. The boxplots of these results are shown in Fig. 9, which illustrates the performance of the tested algorithms.

The results in Table 4 indicate that IM1 (proposed algorithm) had the best R_{nd} and IGD among all combinations. Specifically, for R_{nd} , the values obtained by IM1 were higher than those of the other instances. The IGD values obtained by IM1 were smaller than those obtained by IM2, IM3, and IM4 in each case. Thus, the experimental results based on the R_{nd} and IGD metrics showed that the IMOEA/D produced the most non-dominated solutions, demonstrating the effectiveness of the proposed initialization method. The boxplot in Fig. 9 (a) reflects the solution quality degree of the algorithms, where the smallest R_{nd} of IM1 was near 0.4, surpassing the optimal R_{nd} in the other algorithms among all benchmarks. In addition, the boxplot in Fig. 9 (b) presents the comprehensive performance of the algorithms. It can be seen that the median of IM1 is less than 0.1, whereas the median of the other algorithms is near 0.2. According to the optimization rule of R_{nd} and IGD, the two boxplots in Fig. 9 (b) indicate that the proposed initialization method is the most effective.

Table 5 presents a comparison of the PLS operator results. Regarding R_{nd} , it was apparent that IM1 had a better performance than IM5 in all benchmarks, except benchmark C17. The boxplot of R_{nd} in Fig. 9 (c) indicates that the least acceptable result in IM1 was close to the most promising result in IM5; that is, they were close to 0.5. Furthermore, the

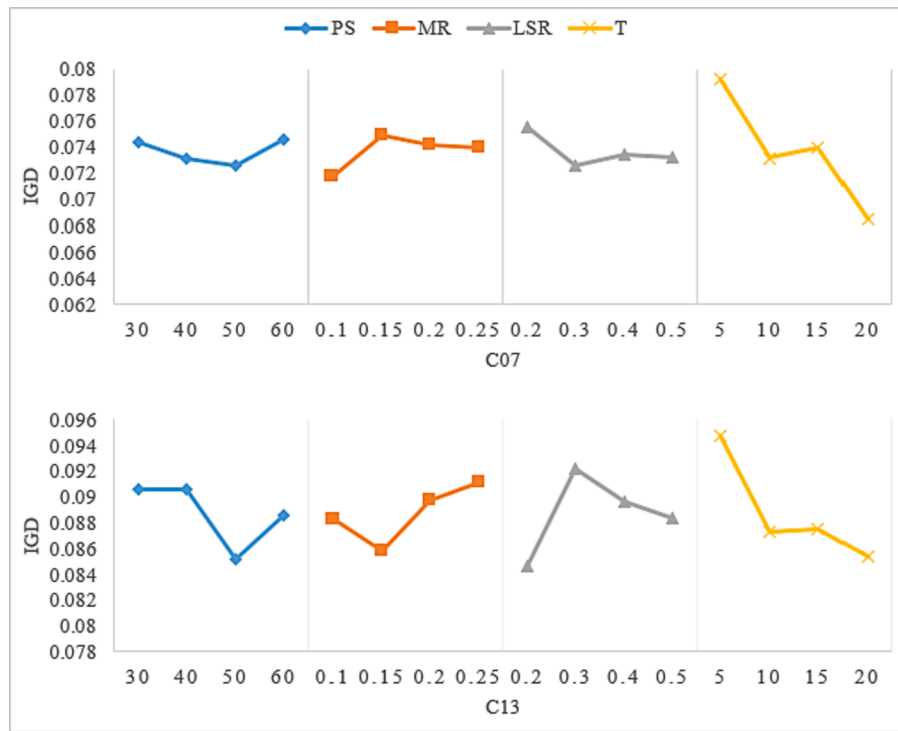


Fig. 8. Performance trend of key parameters.

Table 2

Results of instances C01–C08 run by CPLEX and IMOEA/D.

Ins.	CPLEX			IMOEA/D			Ins.	CPLEX			IMOEA/D		
	makespan	TEC	CPU	makespan	TEC	CPU		makespan	TEC	CPU	makespan	TEC	CPU
C01	557	1005	2.01	581	904	15	C05	–	–	3600	658	2178	30
C02	536	1151	7.43	561	932	22.5	C06	–	–	3600	909	2161	45
C03	824	2131	4.51	910	2011	30	C07	–	–	3600	1112	4523	60
C04	685	2041	98.03	742	1977	45	C08	–	–	3600	1048	4152	90

Table 3

Variants of IMOEA/D.

Algorithms	Initialization methods	Using PLS operator	Using ER operator
IM1 (IMOEA/D1)	Eight heuristics	Yes	Yes
IM2 (IMOEA/D2)	FCFS + FIM	Yes	Yes
IM3 (IMOEA/D3)	FCFS + FEM	Yes	Yes
IM4 (IMOEA/D4)	Random method	Yes	Yes
IM5 (IMOEA/D5)	Eight heuristics	No	Yes
IM6 (IMOEA/D6)	Eight heuristics	Yes	No

median of the R_{nd} value was greater than 0.6 in IM1 but less than 0.4 in IM5, implying that IM1 had a clear advantage over IM5 in handling all instances. For the IGD in Table 5, in addition to benchmarks C15 and C17, IM1 was better than IM5 in all cases. The boxplot of IGD in Fig. 9 (d) indicates that IM1 is better than IM5 because the minimum and median of IGD are derived from IM1 for all benchmarks. Based on the above discussion, it can be concluded that the IMOEA/D with the PLS operator (IM1) performs better than the IMOEA/D without the PLS operator (IM5), demonstrating the superiority of the PLS operator in improving the searching ability of the IMOEA/D.

Table 5 presents the comparison results of the ER operator. In terms of R_{nd} in Table 5, the results of the 20 combinations run by IM1 were significantly better than those produced by IM6, except for benchmarks C17 and C18. For IGD; furthermore, IM1 achieved the best results among all combinations, except for benchmark C20. The above results imply

that IM1 outperforms IM6 on most benchmarks. The boxplots of R_{nd} and IGD are shown in Fig. 9 (e) and Fig. 9 (f), respectively. Fig. 9 (e) shows that the median and optimal values generated by IM1 were better than those derived by IM6. Moreover, Fig. 9 (f) implies that the median and minimum in IM1 were smaller than those in IM6. Thus, IM1 is superior to IM6 for most benchmarks, demonstrating a better number, distribution, and diversity of non-dominated solutions in the IMOEA/D with the ER operator.

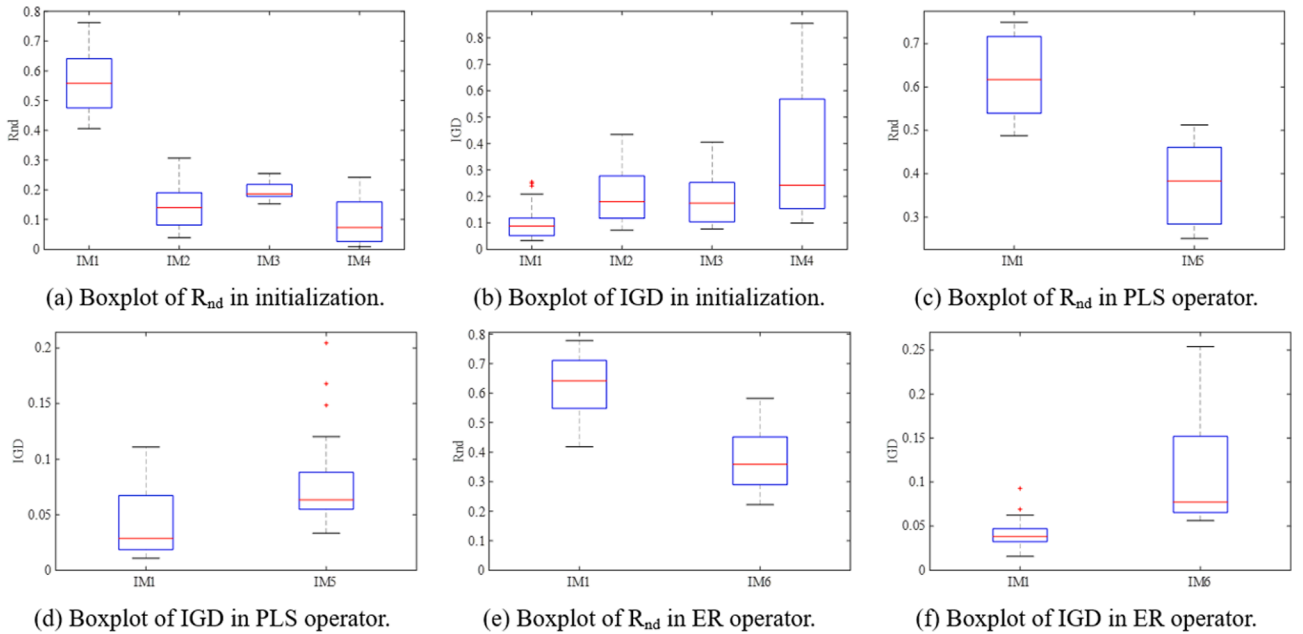
Based on the above experiments and discussions, it can be concluded that the proposed components in the IMOEA/D are effective for solving the DAHFSPT.

5.5. Comparison of algorithms

To further verify the performance of the IMOEA/D, the proposed algorithm was compared with other multi-objective optimization algorithms, namely, NSGA-II (Deb et al., 2002), NNIA (Gong et al., 2008), SPEA-II (Zitzler et al., 2001), NSGA-III (Deb and Jain, 2014) and MOPSO (Coello et al., 2004). These algorithms were chosen because: (1) they are classical population-based algorithms for multi-objective optimization problems, (2) they are widely used as comparison algorithms for scheduling problems, and (3) they are often used to test the performance of a new algorithm. Two hundred instances of 20 combinations were calculated for all the algorithms under the same stopping criteria. For a fair comparison, the original frameworks were maintained for the compared algorithms, and the same encoding, decoding, initialization

Table 4Average IGD and R_{nd} values in comparison of the initialization method.

Comb.	size	R_{nd}				IGD			
		IM1	IM2	IM3	IM4	IM1	IM2	IM3	IM4
C01	$5 \times 5 \times 2 \times 2 \times 3$	0.416356	0.142090	0.199798	0.241756	0.047911	0.109694	0.109969	0.105030
C02	$5 \times 5 \times 2 \times 3 \times 3$	0.407627	0.184008	0.177643	0.234889	0.048714	0.102198	0.098043	0.110701
C03	$5 \times 10 \times 2 \times 2 \times 3$	0.405747	0.182073	0.186397	0.225783	0.057087	0.072067	0.09426	0.099228
C04	$5 \times 10 \times 2 \times 3 \times 3$	0.547751	0.107387	0.210952	0.133910	0.043238	0.091733	0.076673	0.101887
C05	$10 \times 5 \times 2 \times 2 \times 3$	0.490300	0.138326	0.242006	0.129367	0.047107	0.119703	0.131961	0.164141
C06	$10 \times 5 \times 2 \times 3 \times 3$	0.469823	0.213670	0.220772	0.095736	0.054825	0.142775	0.096725	0.171058
C07	$10 \times 10 \times 2 \times 2 \times 3$	0.589328	0.063020	0.162992	0.184660	0.055411	0.114466	0.096649	0.144899
C08	$10 \times 10 \times 2 \times 3 \times 3$	0.654980	0.081190	0.178017	0.085813	0.032432	0.177851	0.174704	0.217973
C09	$20 \times 5 \times 2 \times 2 \times 3$	0.481573	0.181276	0.152547	0.184604	0.112096	0.166063	0.151519	0.183818
C10	$20 \times 5 \times 2 \times 3 \times 3$	0.568032	0.163813	0.223151	0.045004	0.087582	0.182670	0.135412	0.188263
C11	$20 \times 10 \times 2 \times 2 \times 3$	0.626388	0.107598	0.165980	0.100034	0.095471	0.176650	0.173966	0.284291
C12	$20 \times 10 \times 2 \times 3 \times 3$	0.597700	0.086960	0.254728	0.060612	0.064864	0.199250	0.186016	0.266462
C13	$50 \times 5 \times 4 \times 2 \times 6$	0.583886	0.197097	0.210614	0.008403	0.208338	0.214311	0.204504	0.475560
C14	$50 \times 5 \times 4 \times 3 \times 6$	0.465324	0.255678	0.250427	0.028571	0.249084	0.258199	0.262134	0.525809
C15	$50 \times 10 \times 4 \times 2 \times 6$	0.743468	0.082295	0.185226	0.010989	0.113616	0.281165	0.242198	0.585106
C16	$50 \times 10 \times 4 \times 3 \times 6$	0.762104	0.039265	0.184345	0.014286	0.123648	0.272954	0.229391	0.551108
C17	$100 \times 5 \times 4 \times 2 \times 6$	0.527863	0.306066	0.156548	0.009524	0.239603	0.292865	0.310243	0.656715
C18	$100 \times 5 \times 4 \times 3 \times 6$	0.513792	0.275183	0.182453	0.028571	0.254883	0.338613	0.346915	0.676239
C19	$100 \times 10 \times 4 \times 2 \times 6$	0.732148	0.043146	0.181849	0.042857	0.107667	0.372371	0.350088	0.841358
C20	$100 \times 10 \times 4 \times 3 \times 6$	0.704631	0.056777	0.214783	0.023810	0.088572	0.435106	0.405178	0.854773

**Fig. 9.** Boxplots in comparison for proposed components.

and evolutionary strategy were adopted as those in the IMOEA/D. The experimental results based on the R_{nd} and IGD metrics are presented in Table 6 and Table 7. The corresponding boxplots are shown in Fig. 10.

Table 6 shows that the average R_{nd} in the IMOEA/D was larger than that in other algorithms among all instances, except in combinations C04 and C16. NSGA-II and NNIA achieved optimal results only for combinations C04 and C16, respectively. The R_{nd} results indicate that the IMOEA/D outperforms other comparison algorithms in terms of the number of non-dominated solutions. The boxplot of R_{nd} based on Table 6 is shown in Fig. 10 (a), where the median of the IMOEA/D was close to 0.6 and the medians of the other algorithms were less than 0.2. Furthermore, the maximum R_{nd} in the IMOEA/D was close to 1, while it was less than 0.2 in MOPSO, NSGA-III, and SPEA2. Although the maximum values of R_{nd} for NNIA and NSGA-II were close to 0.3, they were much lower than the maximum R_{nd} for the IMOEA/D. Based on the above analysis, the performance in terms of obtaining non-dominated solutions among all algorithms can be ordered as follows: IMOEA/D

> NSGA-II > NNIA > MOPSO > SPEA2 > NSGA-III.

The IGD values in Table 7 are similar to the results of the R_{nd} indicator, that is, the IMOEA/D achieved the best results in all instances, except for cases C01, C02, C04, and C16. Thus, the IMOEA/D has the best comprehensive performance among the six algorithms, including the distribution and diversity of the non-dominated solutions. The boxplot of IGD in Fig. 10 (b) also shows that the IMOEA/D achieved optimal performance and computational stability. Specifically, the median IGD in the IMOEA/D was less than 0.05, but the median IGD in the other algorithms exceeded 0.1. Furthermore, the IMOEA/D generated the shortest range from bottom to top among the six boxes, reflecting the highest computational. Thus, the IMOEA/D has the best IGD performance and stability among the six algorithms.

In summary, compared with five classical multi-objective algorithms, the IMOEA/D obtained the best performance in solving the DAHFSPT, proving the effectiveness of the proposed algorithm.

Fig. 11 shows the PF charts of the small-scale and large-scale

Table 5Average IGD and R_{nd} results in comparison of the PLS.

Comb.	R_{nd}		IGD		R_{nd}		IGD	
	IM1	IM5	IM1	IM5	IM1	IM6	IM1	IM6
C01	0.694619	0.305381	0.016748	0.043593	0.731447	0.268553	0.015823	0.073838
C02	0.532468	0.467532	0.030985	0.03332	0.638236	0.361764	0.032539	0.061502
C03	0.588239	0.411761	0.023584	0.044046	0.555007	0.444993	0.041973	0.057325
C04	0.622291	0.377709	0.034474	0.058017	0.562352	0.437648	0.033204	0.079659
C05	0.749303	0.250697	0.010925	0.059225	0.663946	0.336054	0.032356	0.083719
C06	0.702846	0.297154	0.020336	0.059286	0.745380	0.254620	0.015999	0.067160
C07	0.664427	0.335573	0.016875	0.052293	0.724729	0.275271	0.024799	0.061012
C08	0.674062	0.325938	0.020598	0.057549	0.721324	0.278676	0.020787	0.073440
C09	0.735955	0.264045	0.015554	0.148434	0.667924	0.332076	0.055599	0.063992
C10	0.74238	0.25762	0.012288	0.204461	0.699147	0.300853	0.038791	0.103216
C11	0.545955	0.454045	0.042013	0.057845	0.778037	0.221963	0.032557	0.075260
C12	0.611339	0.388661	0.024108	0.077117	0.670603	0.329397	0.046072	0.056429
C13	0.740979	0.259021	0.032138	0.120335	0.516294	0.483706	0.037573	0.209903
C14	0.729553	0.270447	0.026200	0.167831	0.623695	0.376305	0.048163	0.144643
C15	0.521295	0.478705	0.075184	0.052130	0.530377	0.469623	0.041970	0.171734
C16	0.573369	0.426631	0.065414	0.091268	0.644886	0.355114	0.062450	0.159038
C17	0.487489	0.512511	0.110775	0.067590	0.449206	0.550793	0.039939	0.253864
C18	0.552183	0.447817	0.075866	0.083075	0.417831	0.582168	0.032783	0.249524
C19	0.504504	0.495496	0.069131	0.072166	0.560892	0.439108	0.069151	0.070288
C20	0.520675	0.479325	0.071908	0.085069	0.541172	0.458828	0.092745	0.083152

Table 6Average R_{nd} results in comparison of all algorithms.

Comb.	IMOEA/D	MOPSO	NNIA	NSGA-II	NSGA-III	SPEA2
C01	0.342473	0.077061	0.061242	0.333593	0.030473	0.155158
C02	0.505044	0.057061	0.06824	0.219677	0.022308	0.149978
C03	0.536427	0.108168	0.025132	0.171860	0.020779	0.113463
C04	0.226469	0.091175	0.170547	0.377567	0.016667	0.141746
C05	0.65508	0.037366	0.080051	0.176311	0.009263	0.041929
C06	0.645604	0.031740	0.05834	0.200750	0.012454	0.051112
C07	0.644291	0.064071	0.136032	0.114250	0.014522	0.026835
C08	0.718655	0.042591	0.069762	0.115075	0.026039	0.027878
C09	0.936863	0.024567	0.005000	0.016438	0.011250	0.005882
C10	0.846364	0.040980	0.029771	0.051536	0.011765	0.019583
C11	0.600388	0.097144	0.144740	0.045176	0.085515	0.027038
C12	0.740683	0.029861	0.106013	0.069707	0.026039	0.027696
C13	0.783336	0.037421	0.092084	0.039692	0.022900	0.024567
C14	0.852515	0.028333	0.036978	0.017692	0.031234	0.033248
C15	0.339443	0.130623	0.307977	0.070788	0.067465	0.083704
C16	0.210808	0.150808	0.344192	0.164217	0.065051	0.064924
C17	0.468542	0.073281	0.171850	0.111786	0.050691	0.117628
C18	0.425808	0.081944	0.281661	0.110539	0.065400	0.034646
C19	0.392658	0.156509	0.105330	0.130424	0.051943	0.163136
C20	0.393672	0.132262	0.172715	0.147530	0.051833	0.101987

Table 7

Average IGD results in comparison of all algorithms.

Comb.	IMOEA/D	MOPSO	NNIA	NSGA-II	NSGA-III	SPEA2
C01	0.035592	0.077358	0.085215	0.033663	0.193252	0.044464
C02	0.051585	0.075426	0.072765	0.027173	0.249118	0.045900
C03	0.052476	0.110454	0.061132	0.067466	0.250612	0.071221
C04	0.076355	0.121778	0.136650	0.037222	0.348688	0.065125
C05	0.025505	0.099409	0.077107	0.063707	0.203963	0.098921
C06	0.02482	0.094392	0.088666	0.056168	0.236326	0.084821
C07	0.028461	0.088457	0.065375	0.067175	0.137147	0.100060
C08	0.011339	0.089802	0.082858	0.069778	0.177522	0.107855
C09	0.004249	0.116043	0.119769	0.119016	0.247531	0.176414
C10	0.009288	0.107764	0.133131	0.097434	0.339701	0.206849
C11	0.048048	0.117612	0.092740	0.098986	0.107506	0.137903
C12	0.022670	0.126173	0.110960	0.108683	0.168250	0.151417
C13	0.022073	0.177158	0.128957	0.177928	0.210460	0.218562
C14	0.003456	0.229677	0.217921	0.212891	0.246538	0.243434
C15	0.050516	0.137163	0.184096	0.148754	0.190534	0.179968
C16	0.098846	0.144735	0.076744	0.130539	0.168487	0.192332
C17	0.079051	0.182341	0.213178	0.196967	0.194830	0.191317
C18	0.095129	0.175089	0.245492	0.150522	0.228440	0.247008
C19	0.055731	0.177776	0.171340	0.155167	0.165210	0.190256
C20	0.066944	0.129634	0.186866	0.157736	0.212730	0.197531

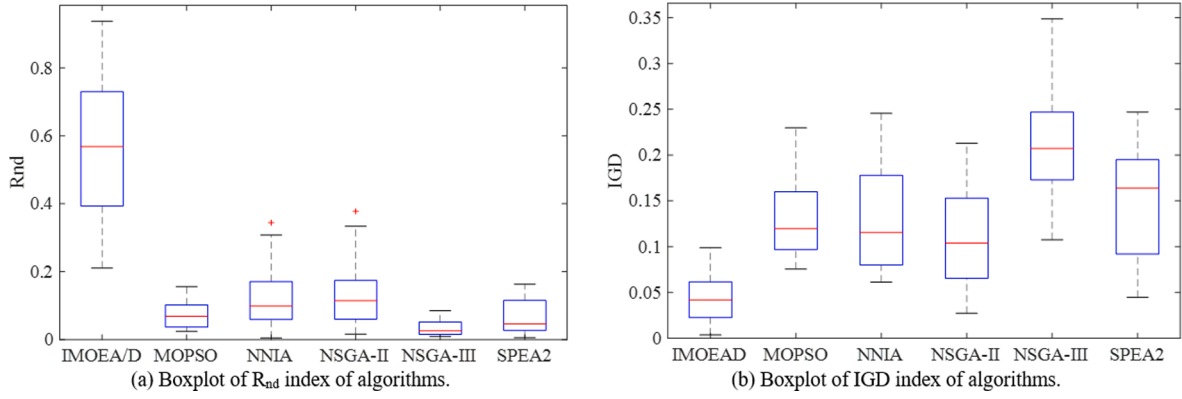


Fig. 10. Boxplots in comparison of all algorithms.

instances for all comparison algorithms. In Fig. 11 (a), the solutions produced by the IMOEA/D were uniformly distributed. They were closer to the coordinate axis than the other algorithms, indicating that the IMOEA/D achieved the best searching ability in depth and breadth. In Fig. 11 (b), it can be seen that the Pareto solutions of the IMOEA/D dominate most of the solutions of the other algorithms. Therefore, the charts in Fig. 11 also reflect the superiority of the IMOEA/D in solving the DAHFSP.

6. Conclusion and future study

In this study, a crucial variant of the distributed assembly flowshop scheduling problem (DAFSP) was first investigated, that is, the distributed assembly hybrid flowshop scheduling problem with transportation resource scheduling (DAHFSP), where a limited number of vehicles in the transportation stage and the early assembly in the assembly stage are involved. A mixed-integer linear programming (MILP) of the DAHFSP was constructed to minimize the makespan and total energy consumption. An improved multi-objective evolutionary algorithm based on decomposition (IMOEA/D) was proposed to address this problem. A four-layer chromosome encoding method was designed to exhibit a complete solution, and the JIT strategy was incorporated into chromosome decoding for saving energy. Eight heuristic rules were developed to generate a high-quality population. The crossover based on the binary matrix and two-point mutation operators were used to perform a global search, and a problem-based local search (PLS) operator was used to improve local exploitation ability. Furthermore, an elimination-reinitialization (ER) operator was designed to avoid falling into local optimal solutions.

Before the experiments, according to the classical instances in the flowshop scheduling problem, this paper formulated twenty

combinations, and each of which includes 10 instances with the same scale. The main parameters of IMOEA/D were calibrated using the Taguchi method of design of experiment. In the experiments, the MILP model of the problem was first verified by CPLEX and compared with the IMOEA/D based on eight small-scale examples. Then, the effectiveness of our well-designed operators was verified, including the initialization, problem-based local search and elimination-reinitialization operators. Subsequently, the effectiveness of the IMOEA/D was evaluated by comparing it with five famous multi-objective optimization algorithms, namely, MOPSO, NNIA, NSGA-II, NSGA-III and SPEA-II. The experimental results showed that the IMOEA/D is significantly better than the comparison algorithms in solving the DAHFSP.

This study has both theoretical and practical significance. Considering the lack of research on the DAFSP with transportation resource scheduling, this study first investigated the DAHFSP and built its MILP. This study could guide scholars to perform wider and deeper research on the DAFSP with transportation resource scheduling. In addition, an effective IMOEA/D was proposed to address the DAHFSP, where the makespan and total energy consumption are minimized simultaneously. The solutions generated by the proposed algorithm can help production managers balance the time- and energy-related goals in the distributed assembly manufacturing units with the transportation stage.

Future studies will focus on other variants of the DAHFSP, such as DAHFSP with batch transportation and energy-efficient DAHFSP, considering machine speed. These problems are in line with realistic manufacturing systems. In addition, the meta-heuristic approaches driven by Q-learning mechanism has been verified to be effective in solving shop scheduling problems, and incorporating this new method into our proposed IMOEA/D to solve the proposed DAHFSP could also be valuable.

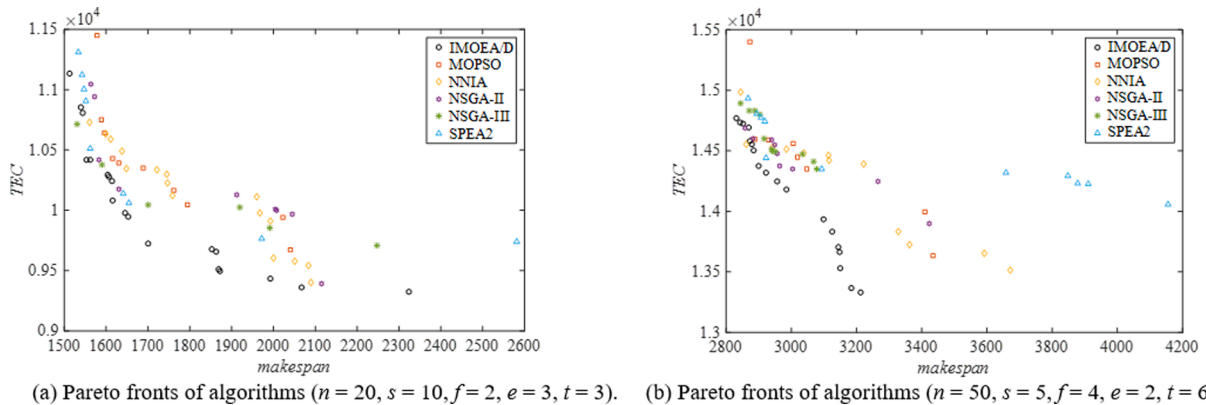


Fig. 11. Pareto fronts in small-scale and large-scale instances.

CRediT authorship contribution statement

Qiang Luo: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. **Qianwang Deng:** Project administration, Supervision, Funding acquisition, Resources. **Xin Guo:** Validation. **Guiliang Gong:** Project administration, Supervision. **Xinhe Zhao:** Validation, Resources. **Liangwu Chen:** Resources.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was funded by the National Key R&D Program of China (No. 2020YFB1712100); Foshan Technological Innovation Project (No. 1920001000041); Postgraduate Scientific Research Innovation Project of Hunan Province (No. CX20230386); Nature Science Foundation of Hunan (No. 2021JJ41081); and Natural Science Foundation of Changsha (No. kq2007033).

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.cie.2023.109717>.

References

- Ahmadian, M. M., Salehipour, A., & Cheng, T. C. E. (2021). A meta-heuristic to solve the just-in-time job-shop scheduling problem. *European Journal of Operational Research*, 288(1), 14–29.
- Cai, J., Lei, D., Wang, J., et al. (2022). A novel shuffled frog-leaping algorithm with reinforcement learning for distributed assembly hybrid flow shop scheduling. *International Journal of Production Research*, 61(4), 1233–1251.
- Cai, J. C., Zhou, R., & Lei, D. M. (2020). Fuzzy distributed two-stage hybrid flow shop scheduling problem with setup time: Collaborative variable search. *Journal of Intelligent & Fuzzy Systems*, 38(3), 3189–3199.
- Coello, C., Pulido, G. T., & Lechuga, M. S. (2004). Handling multiple objectives with particle swarm optimization. *Ieee Transactions on Evolutionary Computation*, 8(3), 256–279.
- Danisa Romero-Ocano, A., Cosio-Leon, M. A., Valenzuela-Alcaraz, V. M., et al. (2022). The impact of gradually replacing fossil fuel-powered vehicles with electric ones: A bi-objective optimisation approach. *Expert Systems with Applications*, 194, Article 116546.
- Deb, K., & Jain, H. (2014). An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I: Solving Problems With Box Constraints. *Ieee Transactions on Evolutionary Computation*, 18(4), 577–601.
- Deb, K., Pratap, A., Agarwal, S., et al. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *Ieee Transactions on Evolutionary Computation*, 6(2), 182–197.
- Deng, J., Wang, L., Wang, S.-Y., et al. (2016). A competitive memetic algorithm for the distributed two-stage assembly flow-shop scheduling problem. *International Journal of Production Research*, 54(12), 3561–3577.
- Elalouf, A. (2021). An FPTAS for just-in-time scheduling of a flow shop manufacturing process with different service quality levels. *Rairo-Operations Research*, 55, S727–S740.
- Gao, K. Z., Suganthan, P. N., Pan, Q. K., et al. (2014). Pareto-based grouping discrete harmony search algorithm for multi-objective flexible job shop scheduling. *Information Sciences*, 289, 76–90.
- Gong, G., Chiong, R., Deng, Q., et al. (2020). A memetic algorithm for multi-objective distributed production scheduling: Minimizing the makespan and total energy consumption. *Journal of Intelligent Manufacturing*, 31(6), 1443–1466.
- Gong, M. G., Jiao, L. C., Du, H. F., et al. (2008). Multiobjective immune algorithm with nondominated neighbor-based selection. *Evolutionary Computation*, 16(2), 225–255.
- Hamzadayi, A., Arvas, M. A., & Elmi, A. (2021). Distributed assembly permutation flow shop problem: Single seekers society algorithm. *Journal of Manufacturing Systems*, 61, 613–631.
- Han, W., Deng, Q., Gong, G., et al. (2021). Multi-objective evolutionary algorithms with heuristic decoding for hybrid flow shop scheduling problem with worker constraint. *Expert Systems with Applications*, 168, Article 114282.
- Hatami, S., Ruiz, R., & Andres-Romano, C. (2013). The Distributed Assembly Permutation Flowshop Scheduling Problem. *International Journal of Production Research*, 51(17), 5292–5308.
- Hatami, S., Ruiz, R., & Andres-Romano, C. (2015). Heuristics and metaheuristics for the distributed assembly permutation flowshop scheduling problem with sequence dependent setup times. *International Journal of Production Economics*, 169, 76–88.
- Huang, J. L., & Gu, X. S. (2021). Distributed assembly permutation flow-shop scheduling problem with sequence-dependent set-up times using a novel biogeography-based optimization algorithm. *Engineering Optimization*.
- Huang, Y.-Y., Pan, Q.-K., & Gao, L. (2022). An effective memetic algorithm for the distributed flowshop scheduling problem with an assemble machine. *International Journal of Production Research*.
- Huang, Y.-Y., Pan, Q.-K., Huang, J.-P., et al. (2021). An improved iterated greedy algorithm for the distributed assembly permutation flowshop scheduling problem. *Computers & Industrial Engineering*, 152, Article 107021.
- Jayaswal, S., & Agarwal, P. (2014). Balancing U-shaped assembly lines with resource dependent task times: A Simulated Annealing approach. *Journal of Manufacturing Systems*, 33(4), 522–534.
- Ji, M., Yang, Y., Duan, W., et al. (2016). Scheduling of No-wait Stochastic Distributed Assembly Flowshop by Hybrid PSO. In *IEEE Congress on Evolutionary Computation (CEC) held as part of IEEE World Congress on Computational Intelligence (IEEE WCCI)* (pp. 2649–2654). Vancouver, CANADA.
- Jia, Y. H., Yan, Q., & Wang, H. F. (2023). Q-learning driven multi-population memetic algorithm for distributed three-stage assembly hybrid flow shop scheduling with flexible preventive maintenance. *Expert Systems with Applications*, 232.
- Jiang, E.-D., & Wang, L. (2019). An improved multi-objective evolutionary algorithm based on decomposition for energy-efficient permutation flow shop scheduling problem with sequence-dependent setup time. *International Journal of Production Research*, 57(6), 1756–1771.
- Jiang, E.-D., Wang, L., & Peng, Z.-P. (2020). Solving energy-efficient distributed job shop scheduling via multi-objective evolutionary algorithm with decomposition. *Swarm and Evolutionary Computation*, 58, Article 100745.
- Jiang, S.-L., & Zhang, L. (2019). Energy-oriented Scheduling for Hybrid Flow Shop With Limited Buffers Through Efficient Multi-Objective Optimization. *Ieee Access*, 7, 34477–34487.
- Jolai, F., Amalnick, M. S., Alinaghian, M., et al. (2011). A hybrid memetic algorithm for maximizing the weighted number of just-in-time jobs on unrelated parallel machines. *Journal of Intelligent Manufacturing*, 22(2), 247–261.
- Lee, C. Y., Cheng, T. C. E., & Lin, B. M. T. (1993). Minimizing the Makespan in the 3-Machine Assembly-Type Flowshop Scheduling Problem. *Management Science*, 39(5), 616–625.
- Lei, D. M., & Dai, T. (2023). A Shuffled Frog-Leaping Algorithm with Cooperations for distributed Assembly Hybrid-Flow Shop Scheduling with Factory Eligibility. *Symmetry-Basel*, 15(4), 786.
- Lei, D. M., & Li, J. L. (2022). Distributed Energy-Efficient Assembly Scheduling Problem with Transportation Capacity. *Symmetry-Basel*, 14(11), 2225.
- Lei, D. M., Su, B., & Li, M. (2021). Cooperated teaching-learning-based optimisation for distributed two-stage assembly flow shop scheduling. *International Journal of Production Research*, 59(23), 7232–7245.
- Lei, D. M., & Wang, T. (2020). Solving distributed two-stage hybrid flowshop scheduling using a shuffled frog-leaping algorithm with memplex grouping. *Engineering Optimization*, 52(9), 1461–1474.
- Li, P., Yang, Y., Du, X., et al. (2017). Iterated Local Search for Distributed Multiple Assembly No-wait Flowshop Scheduling. In *IEEE Congress on Evolutionary Computation (CEC)* (pp. 1565–1571). Spain.
- Li, Q. H., Li, J. Q., Zhang, Q. K., et al. (2021). An improved whale optimisation algorithm for distributed assembly flow shop with crane transportation. *International Journal of Automation and Control*, 15(6), 710–743.
- Li, Y.-Z., Pan, Q.-K., Ruiz, R., et al. (2022). A referenced iterated greedy algorithm for the distributed assembly mixed no-idle permutation flowshop scheduling problem with the total tardiness criterion. *Knowledge-Based Systems*, 239, Article 108036.
- Lin, J., Wang, Z. J., & Li, X. D. (2017). A backtracking search hyper-heuristic for the distributed assembly flow-shop scheduling problem. *Swarm and Evolutionary Computation*, 36, 124–135.
- Lin, J., & Zhang, S. (2016). An effective hybrid biogeography-based optimization algorithm for the distributed assembly permutation flow-shop scheduling problem. *Computers & Industrial Engineering*, 97, 128–136.
- Lu, C., Huang, Y. X., Meng, L. L., et al. (2022). A Pareto-based collaborative multi-objective optimization algorithm for energy-efficient scheduling of distributed permutation flow-shop with limited buffers. *Robotics and Computer-Integrated Manufacturing*, 74, Article 102277.
- Luo, Q., Deng, Q., Gong, G., et al. (2020). An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers. *Expert Systems with Applications*, 160, Article 113721.
- Luo, Q., Deng, Q., Xie, G., et al. (2023). A Pareto-based two-stage evolutionary algorithm for flexible job shop scheduling problem with worker cooperation flexibility. *Robotics and Computer-Integrated Manufacturing*, 82.
- Mainieri, G. B., & Ronconi, D. P. (2013). New heuristics for total tardiness minimization in a flexible flowshop. *Optimization Letters*, 7(4), 665–684.
- Montgomery Douglas C. (2008). *Design and analysis of experiments*. New York: John Wiley and Sons.

- Niu, W., Li, J.-Q., Jin, H., et al. (2022). Bi-objective optimization using an improved NSGA-II for energy-efficient scheduling of a distributed assembly blocking flowshop. *Engineering Optimization*.
- Ong, Y. S., & Keane, A. J. (2004). Meta-Lamarckian learning in memetic algorithms. *Ieee Transactions on Evolutionary Computation*, 8(2), 99–110.
- Pan, Q. K., Gao, L., Li, X. Y., et al. (2019). Effective constructive heuristics and meta-heuristics for the distributed assembly permutation flowshop scheduling problem. *Applied Soft Computing*, 81, Unsp 105492.
- Pourhejazy, P., Cheng, C.-Y., Ying, K.-C., et al. (2021). Supply chain-oriented two-stage assembly flowshops with sequence-dependent setup times. *Journal of Manufacturing Systems*, 61, 139–154.
- Pourhejazy, P., Cheng, C.-Y., Ying, K.-C., et al. (2022). Meta-Lamarckian-based iterated greedy for optimizing distributed two-stage assembly flowshops with mixed setups. *Annals of Operations Research*.
- Shao, W. S., Pi, D. C., & Shao, Z. S. (2019). Local Search Methods for a Distributed Assembly No-Idle Flow Shop Scheduling Problem. *Ieee Systems Journal*, 13(2), 1945–1956.
- Shao, Z. S., Shao, W. S., & Pi, D. C. (2020). Effective Constructive Heuristic and Metaheuristic for the Distributed Assembly Blocking Flow-shop Scheduling Problem. *Applied Intelligence*, 50(12), 4647–4669.
- Song, H. B., & Lin, J. (2021). A genetic programming hyper-heuristic for the distributed assembly permutation flow-shop scheduling problem with sequence dependent setup times. *Swarm and Evolutionary Computation*, 60, Article 100807.
- Taillard, E. D. (1993). Benchmarks for Basic Scheduling Problems. *European Journal of Operational Research*, 64(2), 278–285.
- Wang, J.-J., & Wang, L. (2022). A cooperative memetic algorithm with feedback for the energy-aware distributed flow-shops with flexible assembly scheduling. *Computers & Industrial Engineering*, 168, Article 108126.
- Wang, J., Lei, D. M., & Li, M. B. (2022). A Q-Learning-Based Artificial Bee Colony Algorithm for Distributed Three-Stage Assembly Scheduling with Factory Eligibility and Setup Times. *Machines*, 10(8), 661.
- Wang, J., Tang, H. T., & Lei, D. M. (2023). A Q-learning artificial bee colony for distributed assembly flow shop scheduling with factory eligibility, transportation capacity and setup time. *Engineering Applications of Artificial Intelligence*, 123, Article 106230.
- Wang, L., Wang, J.-J., & Jiang, E. (2021). Decomposition based multiobjective evolutionary algorithm with adaptive resource allocation for energy-aware welding shop scheduling problem. *Computers & Industrial Engineering*, 162, Article 107778.
- Wang, S. Y., & Wang, L. (2016). An Estimation of Distribution Algorithm-Based Memetic Algorithm for the Distributed Assembly Permutation Flow-Shop Scheduling Problem. *Ieee Transactions on Systems Man Cybernetics-Systems*, 46(1), 139–149.
- Wu, X., Peng, J., Xie, Z., et al. (2021). An improved multi-objective optimization algorithm for solving flexible job shop scheduling problem with variable batches. *Journal of Systems Engineering and Electronics*, 32(2), 272–285.
- Xiong, F., Chu, M., Li, Z., et al. (2021). Just-in-time scheduling for a distributed concrete precast flow shop system. *Computers & Operations Research*, 129, Article 105204.
- Xiong, F., & Xing, K. (2014). Meta-heuristics for the distributed two-stage assembly scheduling problem with bi-criteria of makespan and mean completion time. *International Journal of Production Research*, 52(9), 2743–2766.
- Yang, S., & Xu, Z. (2021). The distributed assembly permutation flowshop scheduling problem with flexible assembly and batch delivery. *International Journal of Production Research*, 59(13), 4053–4071.
- Yang, Y., & Li, X. (2022). A knowledge-driven constructive heuristic algorithm for the distributed assembly blocking flow shop scheduling problem. *Expert Systems with Applications*, 202, Article 117269.
- Zhang, G., Xing, K., & Cao, F. (2018). Scheduling distributed flowshops with flexible assembly and set-up time to minimise makespan. *International Journal of Production Research*, 56(9), 3226–3244.
- Zhang, G. H., & Xing, K. Y. (2018). Memetic social spider optimization algorithm for scheduling two-stage assembly flowshop in a distributed environment. *Computers & Industrial Engineering*, 125, 423–433.
- Zhang, Q., & Li, H. (2007). MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *Ieee Transactions on Evolutionary Computation*, 11(6), 712–731.
- Zhang, X., Li, X. T., & Yin, M. H. (2020). An enhanced genetic algorithm for the distributed assembly permutation flowshop scheduling problem. *International Journal of Bio-Inspired Computation*, 15(2), 113–124.
- Zhang, Z.-Q., Hu, R., Qian, B., et al. (2022). A matrix cube-based estimation of distribution algorithm for the energy-efficient distributed assembly permutation flow-shop scheduling problem. *Expert Systems with Applications*, 194, Article 116484.
- Zhang, Z. Q., Qian, B., Hu, R., et al. (2021). A matrix-cube-based estimation of distribution algorithm for the distributed assembly permutation flow-shop scheduling problem. *Swarm and Evolutionary Computation*, 60, Article 100785.
- Zhao, F., Chen, Z., Wang, J., et al. (2017). An improved MOEA/D for multi-objective job shop scheduling problem. *International Journal of Computer Integrated Manufacturing*, 30(6), 616–640.
- Zhao, F., Shao, D., Wang, L., et al. (2022). An effective water wave optimization algorithm with problem-specific knowledge for the distributed assembly blocking flow-shop scheduling problem. *Knowledge-Based Systems*, 243, Article 108471.
- Zhao, F. Q., Zhang, L. X., Cao, J., et al. (2021). A cooperative water wave optimization algorithm with reinforcement learning for the distributed assembly no-idle flowshop scheduling problem. *Computers & Industrial Engineering*, 153, Article 107082.
- Zhou, B., & Wu, Q. (2020). Decomposition-based bi-objective optimization for sustainable robotic assembly line balancing problems. *Journal of Manufacturing Systems*, 55, 30–43.
- Zitzler, E., Laumanns, M., & Thiele, L. J. T. R. G. (2001). SPEA2: Improving the strength pareto evolutionary algorithm.